

Software Practices & Python Craftsmanship — A Practical Guide

SDLC · Agile & Scrum · Version Control · Documentation · Risk Management · Coding Standards & PEP-8 · Comments/Docstrings · Error Handling & Logging · Efficient Code · Design Principles · Unit Testing · Ruff & Black

Prepared for Gitesh — a simple-language reference with examples, diagrams, and quick self-check questions.

Contents

1	Software Development Life Cycle (SDLC)
2	Agile & Scrum
3	Code Version Control
4	Documentation
5	Risk Management
6	Python Coding Standards & PEP-8
7	Comments & Docstrings
8	Error Handling & Logging
9	Writing Efficient Code
10	Design Principles (DRY, KISS, SOLID, YAGNI)
11	Unit Testing & Validation
12	Ruff & Black

1. Software Development Life Cycle (SDLC)

SDLC is the standard set of stages a software project moves through, from idea to a working, maintained product.



Stage	What happens
Requirements	Understand what the client/user actually needs
Design	Plan architecture, data flow, and technology choices
Development	Write the actual code
Testing	Verify the software works as intended and catch bugs
Deployment	Release the software to real users
Maintenance	Fix issues, add improvements after release

Think of it like...Building a house: you don't start laying bricks (development) before you have a blueprint (design), and you don't hand over the keys (deployment) before an inspection (testing).

2. Agile & Scrum

Agile is a mindset for building software in small, frequent steps instead of one giant plan — allowing teams to adapt as requirements change.

Scrum is one popular framework that puts Agile into practice using fixed-length cycles called **Sprints**.

Core Scrum Pieces

Term	Meaning
Sprint	A fixed time box (often 1-4 weeks) to complete a set amount of work
Product Backlog	The full list of features/fixes the product might need, prioritized
Sprint Backlog	The subset of items chosen for the current sprint
Daily Stand-up	A short daily sync: what I did, what I'll do, any blockers
Sprint Review	Demo the completed work at the end of the sprint
Retrospective	Reflect on what went well and what to improve next time
Scrum Master	Helps the team follow the process and removes blockers
Product Owner	Decides priorities based on business/user value

Why teams like itShort cycles mean mistakes or wrong assumptions get caught in weeks, not months — and stakeholders see real progress regularly instead of waiting for one big release.

3. Code Version Control

Version control (most commonly **Git**) tracks every change made to code over time, letting multiple people work on the same project without overwriting each other's work.

```
git init                # start tracking a project
git add file.py         # stage a change
git commit -m "Add login logic" # save a snapshot with a message
git branch feature-x    # create a separate line of work
git checkout feature-x  # switch to it
git merge feature-x     # bring changes back into main
git push origin main    # upload to a remote repo (e.g. GitHub)
```

Think of it like...A video game's save system. Each commit is a save point you can return to; branches let you try a risky move in a separate save file without losing your main progress.

Good habitCommit small, logical chunks with clear messages ("Fix null check in payment flow") rather than one huge commit at the end of the day.

4. Documentation

Documentation explains how software works and how to use it — for future developers, teammates, and even your future self.

Type	Purpose
README	Quick overview: what the project does, how to set it up and run it
API docs	Describes functions/endpoints: inputs, outputs, and behavior
Inline docstrings	Explains individual functions/classes right where they're defined
Architecture docs	Explains the big picture: how components fit together
User guides	Step-by-step instructions for end users

Key idea Good documentation is written for the reader who knows nothing about the project yet — it should let a new person get productive without needing to ask you directly.

5. Risk Management

Risk management means spotting things that could go wrong in a project *before* they happen, and having a plan ready.



Risk type	Example
Technical	A key library becomes unsupported or has a critical bug
Schedule	A dependency team delivers late, delaying your milestone
Resource	A key team member is unavailable (leave, attrition)
Security	An unpatched vulnerability is exploited
Scope	Requirements keep expanding beyond the original plan ("scope creep")

Simple approachFor each risk, rate its *likelihood* and *impact*, and prioritize your attention on risks that are both likely and damaging.

6. Python Coding Standards & PEP-8

PEP-8 is Python's official style guide — a set of conventions that keeps code readable and consistent across different developers and projects.

Core Rules

Rule	Example
Indent with 4 spaces	Not tabs, not 2 spaces
Line length	Max ~79-99 characters per line
Naming: variables/functions	<code>snake_case</code> — e.g. <code>total_price</code>
Naming: classes	<code>PascalCase</code> — e.g. <code>OrderManager</code>
Naming: constants	<code>UPPER_CASE</code> — e.g. <code>MAX_RETRIES</code>
Spacing	<code>x = 1</code> , not <code>x=1</code>
Imports	One per line, grouped: standard library, third-party, local

```
# Good (PEP-8 friendly)
def calculate_total(price, tax_rate=0.18):
    return price + (price * tax_rate)

# Not PEP-8 friendly
def calculateTotal(price,taxRate = 0.18):
    return price+(price*taxRate)
```

Think of it like...Traffic rules. Any car can drive, but everyone following the same rules (which side of the road, what a red light means) is what keeps things safe and predictable for all.

7. Comments & Docstrings

Comments explain tricky bits of logic inline; **docstrings** are a special kind of comment placed right inside functions, classes, or modules that describe their purpose formally.

```
def convert_currency(amount, rate):  
    """  
    Convert an amount from one currency to another.  
  
    Args:  
        amount (float): The amount to convert.  
        rate (float): The conversion rate to apply.  
  
    Returns:  
        float: The converted amount.  
    """  
    return amount * rate  # simple multiplication, rate already includes fees
```

Bonus Tools like `help(convert_currency)` or auto-generated docs (Sphinx) read docstrings automatically — so writing them well pays off beyond just the code file itself.

8. Error Handling & Logging

Error handling deals with problems as they happen (see `try/except`); **logging** is about recording what the program was doing, so you can investigate issues later — especially useful in production, where you can't just watch the program run live.

```
import logging

logging.basicConfig(level=logging.INFO, filename="app.log")

def process_order(order_id):
    logging.info(f"Processing order {order_id}")
    try:
        # ... processing logic ...
        logging.info(f"Order {order_id} completed successfully")
    except Exception as e:
        logging.error(f"Order {order_id} failed: {e}")
```

Log level	When to use
DEBUG	Detailed info, useful only while developing
INFO	Confirms things are working as expected
WARNING	Something unexpected, but the program still works
ERROR	A real problem — something failed
CRITICAL	A severe error — the program may not be able to continue

Avoid Using `print()` for tracking issues in real applications — logs can be leveled, filtered, saved to files, and searched, while prints just clutter the console.

9. Writing Efficient Code

Efficient code uses less time and memory to do the same job — important as data or usage scales up.

```
# Inefficient: builds a list just to sum it
total = sum([x for x in range(1000000)])

# Efficient: generator expression avoids building the whole list in memory
total = sum(x for x in range(1000000))

# Inefficient: repeated string concatenation
result = ""
for word in words:
    result += word

# Efficient: join is much faster for many strings
result = "".join(words)
```

Quick habits

- Use built-in functions (they're implemented in fast C code)
- Avoid unnecessary loops when a list/dict comprehension will do
- Use appropriate data structures — e.g. a **set** for fast membership checks instead of a **list**
- Profile before optimizing — fix the part that's actually slow, not a guess

10. Design Principles (DRY, KISS, SOLID, YAGNI)

Principle	Meaning
DRY	Don't Repeat Yourself — avoid duplicating logic; extract it into one reusable place
KISS	Keep It Simple, Stupid — prefer the simplest solution that works over a clever, complex one
YAGNI	You Aren't Gonna Need It — don't build features "just in case"; build what's actually needed now
SOLID	Five principles (below) for designing maintainable, flexible classes

SOLID, broken down

Letter	Principle	Plain meaning
S	Single Responsibility	A class should have one reason to change — one job
O	Open/Closed	Open for extension, closed for modification — add new behavior without rewriting old code
L	Liskov Substitution	A subclass should be usable anywhere its parent class is, without breaking things
I	Interface Segregation	Don't force a class to implement methods it doesn't need
D	Dependency Inversion	Depend on abstractions (interfaces), not on concrete implementations

Think of it like...DRY is not copy-pasting the same paragraph across ten documents; KISS is choosing stairs over an elevator with 20 buttons when you only need one floor.

11. Unit Testing & Validation

Unit tests check that small, individual pieces of code (usually single functions) behave correctly — automatically, and every time you make a change.

```
import unittest

def add(a, b):
    return a + b

class TestMath(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-2, -3), -5)

if __name__ == "__main__":
    unittest.main()
```

Why it matters Tests act as a safety net: when you change code later, running the test suite quickly tells you if you broke something — without manually re-checking everything by hand.

Quick check: If a test named `test_add_negative_numbers` fails after a code change, what does that tell you?

Answer: Something in your latest change likely broke the logic for adding negative numbers specifically — the test name tells you exactly where to look.

12. Ruff & Black

These are two popular Python tools that keep code clean automatically, without manual effort.

Tool	What it does
Black	A code <i>formatter</i> — automatically rewrites your code's spacing, line breaks, and quotes into one consistent style
Ruff	A very fast <i>linter</i> — scans code for style violations, unused imports, likely bugs, and PEP-8 issues

```
# Install
pip install black ruff

# Format all files in the project automatically
black .

# Check code for issues
ruff check .

# Auto-fix what Ruff safely can
ruff check . --fix
```

Common workflowRun Black to format, then Ruff to catch anything formatting alone can't fix (like unused variables) — many teams run both automatically before every commit.

Think of it like...Black is a spell-checker that automatically fixes typos in formatting; Ruff is a proofreader that flags actual issues in the writing itself (repetition, unused sentences, logic gaps).